

Machine Learning

Chapter 2: Concept Learning and the General-to-Specific Ordering

Slides based on Tom M. Mitchell, *Machine Learning*
(McGraw-Hill, 1997)

Concept Learning as Search

FIND-S: Finding a Maximally Specific Hypothesis

Version Spaces and Candidate Elimination

Inductive Bias

Concept Learning as Search

The concept learning task

Fact (Definition)

Concept learning: *inferring a boolean-valued function from training examples of its input and output.*

We seek to learn a target concept $c : \mathcal{X} \rightarrow \{0, 1\}$ (“yes” / “no”) from a set of training examples of the form $\langle x, c(x) \rangle$, choosing hypotheses from a hypothesis space $\mathcal{H}$.

Running example: EnjoySport

Learn to predict the value of *EnjoySport* for a person, i.e. “does my friend enjoy her favorite water sport today,” from attributes of the day: Sky, AirTemp, Humidity, Wind, Water, Forecast.

Training data: EnjoySport

Ex.	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Hypothesis representation: a conjunction of constraints on the six attributes. Each constraint is

- a specific value (e.g., Water = Warm),
- “?” — any value is acceptable, or
- “ $\emptyset$ ” — no value is acceptable (the concept is never satisfied).

E.g. $h = \langle \text{Sunny}, ?, ?, \text{Strong}, ?, \text{Same} \rangle$.

The inductive learning hypothesis

We only know whether h agrees with c on the *training* examples; the inductive learning task is really to determine h over the entire set of instances $\mathcal{X}$.

Fact (The inductive learning hypothesis)

Any hypothesis found to approximate the target function well over a sufficiently large set of training examples will also approximate the target function well over other unobserved examples.

Notation: most general hypothesis $\langle ?, ?, ?, ?, ?, ? \rangle$ (accepts every instance); most specific hypothesis $\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$ (rejects every instance).

Concept learning as search

Concept learning can be viewed as the task of searching a (usually huge) hypothesis space $\mathcal{H}$ for the hypothesis that best fits the training examples. For EnjoySport, with 3 attributes taking 2 values and 3 taking 3 values (plus “?” and “ $\emptyset$ ”), there are $5 \cdot 4 \cdot 4 \cdot 4 \cdot 4 \cdot 4 = 5120$ syntactically distinct hypotheses, but only $1 + (4 \cdot 3 \cdot 3 \cdot 3 \cdot 3 \cdot 3) = 973$ *semantically* distinct ones once $\emptyset$ -containing hypotheses are merged.

Fact (General-to-specific ordering)

For hypotheses h_j, h_k defined over $\mathcal{X}$: h_j is **more_general_than_or_equal_to** h_k (written $h_j \geq_g h_k$) iff $\forall x \in \mathcal{X} : [(h_k(x) = 1) \rightarrow (h_j(x) = 1)]$.

This partial order lets algorithms navigate $\mathcal{H}$ *without* enumerating it explicitly — e.g., by generalizing or specializing one hypothesis at a time.

FIND-S: Finding a Maximally Specific Hypothesis

The FIND-S algorithm

Algorithm 1: FIND-S

Initialize: $h \leftarrow$ the most specific hypothesis in $\mathcal{H}$

```
1 foreach positive training instance  $x$  do
2   foreach attribute constraint  $a_i$  in  $h$  do
3     if the constraint  $a_i$  in  $h$  is satisfied by  $x$  then
4       | do nothing
5     else
6       | replace  $a_i$  in  $h$  by the next more general constraint satisfied by  $x$ 
```

Output: hypothesis h

FIND-S *ignores negative examples entirely* and simply climbs the general-to-specific ordering, generalizing just enough at each positive example to keep covering it.

Tracing FIND-S on EnjoySport

Step	Hypothesis h
h_0	$\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$
after x_1 (Yes)	$\langle \text{Sunny, Warm, Normal, Strong, Warm, Same} \rangle$
after x_2 (Yes)	$\langle \text{Sunny, Warm, ?, Strong, Warm, Same} \rangle$
x_3 (No)	ignored
after x_4 (Yes)	$\langle \text{Sunny, Warm, ?, Strong, ?, ?} \rangle$

Unanswered questions

- Has the algorithm converged to *the* target concept, even though it never checks negative examples for consistency?
- Why prefer the *maximally specific* hypothesis — are there other hypotheses equally consistent with the data?
- What if the training data is inconsistent (noisy)?
- What if there are several maximally specific consistent hypotheses?

Version Spaces and Candidate Elimination

Fact (Definitions)

A hypothesis h is **consistent** with a set of training examples D iff $h(x) = c(x)$ for every $\langle x, c(x) \rangle \in D$.

The **version space**, $VS_{\mathcal{H}, D}$, is the subset of hypotheses from $\mathcal{H}$ consistent with the training examples in D :

$$VS_{\mathcal{H}, D} \equiv \{h \in \mathcal{H} \mid \text{Consistent}(h, D)\}$$

A naive LIST-THEN-ELIMINATE algorithm initializes VS to *all* of $\mathcal{H}$ and removes any h inconsistent with each new example — correct, but requires exhaustively enumerating $\mathcal{H}$, which is intractable except for the smallest problems.

A compact representation: the S and G boundaries

Every member of the (possibly huge) version space is bracketed between two small *boundary sets*:

- G — the set of **maximally general** members of $VS_{\mathcal{H},D}$
- S — the set of **maximally specific** members of $VS_{\mathcal{H},D}$

Fact (Version space representation theorem)

$$VS_{\mathcal{H},D} = \{h \in \mathcal{H} \mid \exists s \in S, \exists g \in G, g \geq_g h \geq_g s\}$$

So S and G , together with the ordering, implicitly represent every hypothesis in the version space, without listing them.

The CANDIDATE-ELIMINATION algorithm (1/2)

Algorithm 2: CANDIDATE-ELIMINATION, part 1

Initialize: $G \leftarrow \{\text{the most general hypothesis in } \mathcal{H}\}$; $S \leftarrow \{\text{the most specific hypothesis in } \mathcal{H}\}$

```
1 foreach training example  $d$  do
2   if  $d$  is a positive example then
3     remove from  $G$  any hypothesis inconsistent with  $d$ ;
4     foreach hypothesis  $s \in S$  inconsistent with  $d$  do
5       remove  $s$  from  $S$ ;
6       add to  $S$  all minimal generalizations  $h$  of  $s$  such that  $h$  is consistent
          with  $d$ , and some member of  $G$  is more general than  $h$ ;
7       remove from  $S$  any hypothesis more general than another in  $S$ ;
    //  $d$  is a negative example (continued next slide)
```

The CANDIDATE-ELIMINATION algorithm (2/2)

Algorithm 3: CANDIDATE-ELIMINATION, part 2

```
1 foreach training example  $d$ , if  $d$  is negative do
2   remove from  $S$  any hypothesis inconsistent with  $d$ ;
3   foreach hypothesis  $g \in G$  inconsistent with  $d$  do
4     remove  $g$  from  $G$ ;
5     add to  $G$  all minimal specializations  $h$  of  $g$  such that  $h$  is consistent with  $d$ ,
        and some member of  $S$  is more specific than  $h$ ;
6   remove from  $G$  any hypothesis less general than another in  $G$ ;
```

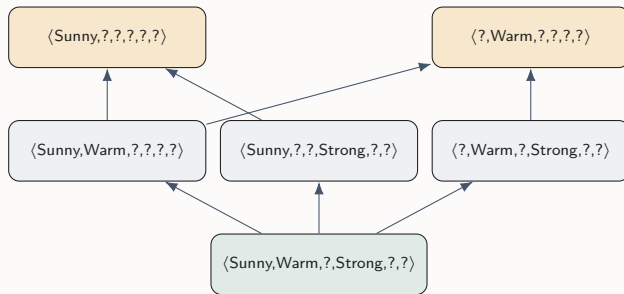
Positive examples generalize S (and prune G); negative examples specialize G (and prune S). The algorithm converges when $S = G = \{h\}$ (a unique consistent hypothesis) or $S = G = \emptyset$ (no consistent hypothesis exists in $\mathcal{H}$).

Trace: S and G boundaries after each EnjoySport example

After	S (specific boundary)	G (general boundary)
d_0 (init)	$\{\langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle\}$	$\{\langle ?, ?, ?, ?, ?, ? \rangle\}$
d_1 (+)	$\{\langle \text{Sunny}, \text{Warm}, \text{Normal}, \text{Strong}, \text{Warm}, \text{Same} \rangle\}$	$\{\langle ?, ?, ?, ?, ?, ? \rangle\}$
d_2 (+)	$\{\langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, \text{Warm}, \text{Same} \rangle\}$	$\{\langle ?, ?, ?, ?, ?, ? \rangle\}$
d_3 (-)	<i>unchanged</i>	$\{\langle \text{Sunny}, ?, ?, ?, ?, ? \rangle, \langle ?, \text{Warm}, ?, ?, ?, ? \rangle, \langle ?, ?, ?, ?, ?, \text{Same} \rangle\}$
d_4 (+)	$\{\langle \text{Sunny}, \text{Warm}, ?, \text{Strong}, ?, ? \rangle\}$	$\{\langle \text{Sunny}, ?, ?, ?, ?, ? \rangle, \langle ?, \text{Warm}, ?, ?, ?, ? \rangle\}$

After all four examples, S and G still bracket *several* equally plausible hypotheses — more training data (or queries) would be needed to converge to a single one.

Picturing the version space



■ S boundary (most specific) ■ G boundary (most general) arrows point from specific to more-general (all consistent with D)

What training example should the learner request next?

An active learner can query for the classification of the instance that *best discriminates* among remaining hypotheses: pick an instance satisfied by roughly half of the hypotheses currently in the version space. Whatever the answer, the version space shrinks by (up to) half — an optimal, generalized binary search over $\mathcal{H}$. **Classifying new instances with a partially learned concept:** an instance can be classified confidently once every hypothesis in the (still-uncollapsed) version space agrees on it; a vote over S , G , and intermediate hypotheses gives a natural confidence score otherwise.

Inductive Bias

A biased hypothesis space

Restricting $\mathcal{H}$ to conjunctions of constraints (as above) means the learner *cannot even represent* some concepts, e.g. “Sky=Sunny or Sky=Cloudy” cannot be described by a single conjunctive hypothesis. If the true concept is not in $\mathcal{H}$, CANDIDATE-ELIMINATION converges to an *empty* version space even though a perfectly good rule exists — just not one expressible in $\mathcal{H}$.

Fact (A fix: an unbiased hypothesis space)

Let $\mathcal{H}'$ be the power set of $\mathcal{X}$ (every possible subset, i.e. every boolean function) so every concept is representable.

The futility of bias-free learning

With the unbiased $\mathcal{H}'$, CANDIDATE-ELIMINATION is guaranteed to converge to a version space containing *exactly* the hypotheses consistent with the observed data — but that version space **never shrinks below** a disjunction that leaves every unseen instance's classification split evenly between remaining hypotheses. **An unbiased learner cannot generalize beyond the observed examples.** Some bias — some prior assumption beyond the data itself — is logically necessary for inductive generalization.

Fact (Definition)

Consider a concept learning algorithm L for the set of instances $\mathcal{X}$. Let c be an arbitrary target concept, $D_c = \{\langle x, c(x) \rangle\}$ its training data, and $L(x_i, D_c)$ the classification L assigns to new instance x_i after training on D_c . The **inductive bias** of L is any minimal set of assertions B such that, for any target concept c and instances D_c :

$$\forall x_i \in \mathcal{X} : (B \wedge D_c \wedge x_i) \vdash L(x_i, D_c)$$

I.e., the bias is the extra assumption that lets deductive inference reproduce the learner's inductive prediction.

Comparing inductive biases

Learner	Inductive bias
Rote learner	No bias — stores examples, never generalizes to unseen instances (weakest possible bias).
CANDIDATE-ELIMINATION	$c \in \mathcal{H}$ (target concept can be represented as a conjunction of attribute constraints).
FIND-S	$c \in \mathcal{H}$, <i>and</i> all instances are negative unless entailed to be positive by the training data (a stronger bias).

Stronger bias $\Rightarrow$ more generalization from less data, but greater risk of ruling out the true concept if the bias is wrong. This bias/variance-style tradeoff recurs throughout the book.

Chapter summary

- Concept learning is search through a partially ordered hypothesis space, guided by the more-general-than relation.
- FIND-S finds a maximally specific consistent hypothesis, cheaply, but ignores negative examples and hides alternative consistent hypotheses.
- CANDIDATE-ELIMINATION maintains the full version space compactly via its S and G boundary sets, using positive examples to generalize S and negative examples to specialize G .
- Active/optimal query strategies can shrink the version space by roughly half per query.
- Every learner that generalizes carries an *inductive bias*; a completely unbiased learner cannot generalize at all.

Source: T. Mitchell, *Machine Learning*, McGraw-Hill, 1997, Chapter 2.